

[TOC]

编译原理研讨课CACT实验PR003报告

成员组成

第03小组：

杜璵 俞博涵 周波

任务说明

前面两次实验中已经完成了对cact的词法分析、语法分析、语义分析和中间代码的生成，在此基础上完成从中间代码生成RISCV汇编代码。

实验过程

实验环境

我们在虚拟机上安装了交叉编译riscv64-unknown-linux-gnu-gcc来编译生成的汇编文件，安装qemu来模拟运行编译完成的可执行文件。下面是我们编译运行使用的命令。其中libmylib.a是编译好的库文件，由runtime.c编译而来，包含getint等CACT内置的IO函数。echo \$?用于打印返回值。

```
./compiler ../test/functional/$num.cact  
riscv64-unknown-linux-gnu-gcc -c $num.S -o $num.o  
riscv64-unknown-linux-gnu-gcc $num.o -L. -lmylib -o $num -static  
qemu-riscv64 ./ $num; echo $?
```

设计思路

RISCV汇编代码生成

- 变量类型识别与加载 (var_to_reg) 把 IR 中的变量名转换为 RISC-V 可用的值，存入寄存器中。
- 指令选择与翻译 将 IR 表达的操作转成具体的RISC-V指令，使用 IRop 字段识别操作类型，如IROP_ADD、IROP_MUL等；判断是否是浮点类型 (basety == BTY_FLOAT)，选择对应的fadd.s, fmul.s；拼接RISC-V指令字符串，加入ASM_array
- 写回变量结果 (reg_to_var) 将计算结果从寄存器写回变量(栈或全局内存)，再次通过前缀判断写入位置，使用 sw, sb, fsw 等指令，根据类型和偏移地址生成代码。
- 特殊结构与数据段生成 对无法直接用立即数加载的值（特别是浮点数和数组初值），放入 .rodata 或 .data 段中，使用 Const_Stack 收集字符串：如 .word, .zero, .float；另外还自动生成标签名 IMM0, IMM1, ... 对应浮点常量。
- 统一汇编输出 将收集的所有指令输出到 RISC-V 汇编文件中：将 ASM_array 中指令输出到.text 段；将 Const_Stack 输出到.rodata；数组初值等输出到.data 段。

代码实现

汇编代码生成

在 `RiscvGen.h` 中定义一个 `RiscvGen` 类，用于处理由中间代码生成汇编代码的过程。main 函数中会实例化一个 `RiscvGen` 类的对象，通过执行它的 `Gen_All` 成员函数来生成所有汇编指令并将其写入文件。

在 `RiscvGen.cpp` 中给出了 `RiscvGen` 类各成员函数的具体定义。

Gen_All

`Gen_All` 函数先向 `ASM_array` 中添加文件名等开头信息，接着声明两个变量：`rescan` 用于标记当前是第一次还是第二次扫描，`rescan_loc`，用于记录重新开始扫描的位置。

随后遍历存放中间代码的数组 `ir_optim.IRC_array`，进行两次扫描。

第一次扫描 (`rescan == false`)：只执行函数栈帧分析 (`Cnt_Temp_Lvar`, `Cnt_FuncParam`)，并跳过非必要生成；

```
case IR_G_ALLOC:
    Gen_G_Alloc(irc); // 全局变量分配
    break;
case IR_L_ALLOC:
    Cnt_Temp_Lvar(irc.result); // 统计局部变量
    break;
case IR_FUNC_BEGIN:
    frame_size = 16; // 初始化栈帧大小
    Cnt_FuncParam(irc.result); // 统计函数参数
    rescan_loc = i; // 记录位置以便后续重新生成
    break;
case IR_FUNC_END:
    frame_size = up_to_align(frame_size, 16); // 对齐栈帧
    rescan = true;
    i = rescan_loc - 1; // 跳回 FUNC_BEGIN 前的位置
    break;
```

第二次扫描 (`rescan == true`)：进入实际的汇编代码生成部分,每种 IR 类型会调用对应的 `Gen_*` 函数生成指令并写入 `ASM_array`:

```
switch(irc.IRop) {
case IR_G_ALLOC: exit(1); // 应该不出现
case IR_L_ALLOC: Gen_L_Alloc(irc); break;
case IR_LABEL: Gen_Label(irc); break;
case IR_FUNC_BEGIN: Gen_FuncBegin(irc); break;
case IR_FUNC_END:
    Gen_FuncEnd(irc);
    rescan = false;
    break;
case IR_PARAM: Gen_Param(irc); break;
```

```

    case IR_CALL: Gen_Call(irc); break;
    case IR_RETURN: Gen_Return(irc); break;
    case IR_ASSIGN: Gen_Assign(irc); break;
    case IR_ADD: case IR_SUB: ... : Gen_Operation(irc); break;
    case IR_BEQ: case IR_BNE: ... : Gen_Branch(irc); break;
    case IR_J: Gen_Jump(irc); break;
    default: exit(1); // 未预期操作
}

```

stack_initval

将数组或变量的初始化值 (initval_str) 转换为 RISC-V 汇编中的静态数据段表示形式 (.byte / .word / .zero) 并存入 Const_Stack，用于后续汇编文件生成。

首先处理初始值字符串，分割初始化字符串

```

//将初始值字符串拆分
std::vector<std::string> initval_array;
//去除前缀
initval_str = initval_str.substr(1);
int loc; //第一个逗号位置
for(int i=0;i<len;i++){
    loc = initval_str.find(',');
    if(i==len-1 && loc==-1){
        //允许最后一个没有逗号，后缀
        initval_array.push_back(initval_str);
    }
    else{
        initval_array.push_back(initval_str.substr(0,loc));
        initval_str = initval_str.substr(loc+1);
    }
}
}

```

然后遍历子串，根据基本类型 basety 的不同，生成相应的汇编指令。

```

if(basety == BTY_BOOL) {
    Const_Stack.push_back(".byte " + initval); // 1字节
}
else if(basety == BTY_INT) {
    Const_Stack.push_back(".word " + initval); // 4字节
}
else if(basety == BTY_FLOAT) {
    float fval = std::stof(initval);
    int decimal_fval = *(int *)&fval; // float转IEEE-754表示的整数
    Const_Stack.push_back(".word " + std::to_string(decimal_fval));
}
}

```

在此过程中，变量 zero_size 用于统计空白位置的大小，每次将其写入 Const_Stack 后清零。

```
size_t base_size = (typeutils.basety_to_size)[basety];
size_t zero_size = 0;
```

var_to_reg

根据变量的命名规则和作用域（全局/局部/临时/立即数），生成适当的 RISC-V 汇编指令，将变量值装载到目标寄存器 `reg_name` 中。

首先解析变量种类。通过变量名的前缀，可以判断变量是否为全局变量、局部变量、临时变量和立即数。

```
bool is_gvar = var_name[0] == GVAR_PREFIX;
bool is_lvar = var_name[0] == LVAR_PREFIX;
bool is_temp = var_name[0] == TEMP_PREFIX;
bool is_imm = var_name[0] == IMM_PREFIX;
```

接着递归处理地址/数组项,识别变量的编号,根据类型生成指令分支处理：`is_gvar` / `is_lvar` / `is_temp` / `is_imm`; 获取基地址或偏移地址（如 `.word -16(s0)`）选择指令：`lb`（1字节）：`bool/char lw`（4字节）：`int flw`（浮点）：`float ld`：加载指针

reg_to_var

将寄存器 `reg_name` 中的值，根据变量名 `var_name` 表示的位置（局部/全局/临时变量，或数组项），生成 RISC-V 汇编指令将值写入内存。

先确定变量的类型，通过前缀判断变量是否为全局变量、局部变量、临时变量。然后在变量名中查找 `ITEM_INFIX`，若有则是数组项，将变量转移到 `s5` 寄存器中。接着分别考虑全局变量、局部变量、临时变量的情况，根据变量的类型用 `fsw`、`fsw`、`sw`、`sb` 等指令将值存入栈中。识别代码如下：

```
bool is_gvar = var_name[0] == GLOBAL_PREFIX;
bool is_lvar = var_name[0] == LOCAL_PREFIX;
bool is_temp = var_name[0] == TEMP_PREFIX;
int item_infix_loc = var_name.find(ITEM_INFIX);
bool is_item = item_infix_loc != -1;
```

Gen_FuncBegin

该函数在第二遍扫描到函数开头时被调用，为中间代码中的函数定义 `irc.result` 生成 RISC-V 汇编函数开头，包括栈帧设置和参数入栈。。

它会先生成开头的信息，然后生成函数名对应的 `label`。

```
ASM_array.push_back(".text");
ASM_array.push_back(".align 1");
ASM_array.push_back(".globl " + func_name);
ASM_array.push_back(".type " + func_name + ", @function");
ASM_array.push_back(func_name + ":" );
```

再根据第一遍扫描时统计的栈帧大小，生成开辟栈帧的指令。随后，它通过函数名在符号表中查找函数的参数列表，将函数形参从寄存器转移到栈上，当前只考虑a0-a7和fa0-fa7。

```
ASM_array.push_back("addi    sp, sp, -" + std::to_string(frame_size));
ASM_array.push_back("sd      ra, " + std::to_string(frame_size-8) + "(sp)");
ASM_array.push_back("sd      s0, " + std::to_string(frame_size-16) + "(sp)");
ASM_array.push_back("addi    s0, sp, " + std::to_string(frame_size));
```

对于整数参数（a0-a7），使用s寄存器和lw、lb指令；整数参数（a0-a7），使用s寄存器和lw、lb指令。最后检查函数参数的数量，如果整数参数数量icnt或浮点数参数数量fcnt超过等于8，则输出错误信息并退出程序。

Gen_FuncEnd

该函数在第二遍扫描到函数结尾时被调用，为当前函数生成退出时的 RISC-V 汇编，包括恢复栈帧、跳转回调用者、记录函数大小。函数恢复：

```
ASM_array.push_back("ld      ra, " + std::to_string(frame_size-8) + "(sp)");
ASM_array.push_back("ld      s0, " + std::to_string(frame_size-16) + "(sp)");
ASM_array.push_back("addi    sp, sp, " + std::to_string(frame_size));
ASM_array.push_back("jr      ra");
```

Gen_G_Alloc

该函数用于生成分配和初始化全局变量的汇编代码，会在第二遍扫描到全局变量时被调用。

首先，根据全局变量的名称获取其在全局变量数组中的索引order和堆栈标签stack_label。获取索引需要从变量名中去除前缀，并转为数字，这一过程通过函数num_from_name实现，使代码更简洁。

随后，根据全局变量的属性判断其所属的分区（.rodata、.data 或 .bss），并将相应的信息添加到Const_Stack中。如果全局变量需要初始化，则调用 stack_initval 函数来解析初始值字符串并生成相应的汇编代码。否则，添加指令.zero来分配适当大小的空间。

Gen_L_Alloc

Gen_L_Alloc函数在第二遍扫描到局部变量时被调用，为全局变量生成 RISC-V 汇编代码（定义在.data/.bss/.rodata 段），支持是否初始化、是否为常量、是否为数组。

首先检查变量是否为局部变量且是否需要进行初始化。如果不是局部变量或不需要初始化，则直接返回。

随后调用num_from_name获取变量的索引order，再获取长度len和偏移量offset。然后在.rodata段声明常量，分别考虑len>1和len<=1两种情况，对数组均使用标号初始化，对浮点的单个常量也需要给出标号。

```
std::string align_str =
    (len != 1) ? ".align 3" : // 数组：按 8 字节对齐
```

```
(basety != BTY_BOOL && basety != BTY_CHAR) ? ".align 2" : // 非bool/char变量  
"";
```

最后，生成将局部变量从标号转移到栈上的汇编代码。对于长度大于1的数组，使用寄存器 `s2` 存储标号的地址，并根据每次转移的字节数选择相应的指令进行转移，在转移8/4/2/1字节时分别采用不同的后缀d/w/h/b。每次采用能转移尽可能多字节的指令，避免不必要的多次转移。对于长度为1的变量，根据基本类型选择相应的指令将初始值存储到栈上。

总结

实验结果总结

本实验中，我们根据中间代码生成汇编代码。整个RISC-V变量生成过程以作用域和类型为核心，配合中间变量命名规范，对变量进行统一抽象编码处理。汇编生成过程中充分考虑了RISC-V的调用约定（如s0为帧指针、全局地址通过la加载）、类型对应的指令选择（如flw对应float类型），并为数组和指针提供了灵活的偏移支持，保证了变量访问在汇编层面的一致性和正确性。这个过程中stack_initval、var_to_reg、reg_to_var作为变量处理的基础函数，是连接语义分析结果与汇编生成之间的重要桥梁。

编译运行所有functional的32个样例都成功通过。编译好的汇编文件、目标文件和可执行文件见于目录functional_asm下。

分成员总结

杜璁：

汇编文件的生成结合之前的实验让我对编译器工作过程有了较为完整的了解，对汇编代码的认识也更多。本实验中我参与代码编写，参与了部分debug，并测试了funcitonal中的样例。

俞博涵：

本次实验中我参与了实验环境的配置，编写了IO内置库，并参与了bug的修改，对riscv的编写和运行过程有了更深入的了解。

周波：

本次实验需要要对中间代码和目标汇编代码之间的对应关系有清晰的规划。通过本次实验，我对编译器生成汇编代码的过程有了更清晰的认识，也巩固了对RISC-V汇编以及ABI的认识，对于之前并不十分了解的伪指令也有了一定的认识，并进一步加深了对编译器工作方式的理